14. Prosperity vs TIDES on one example

Both designs compute the same spiking GEMM: for every filter f, token p and timestep t, **out[f][p][t] = Σ_k W[f][k] · s[p][k][t]**, where s is a spike (0 or 1). The example below is small enough to trace by hand: 2 filters, 4 inputs, 4 tokens, T = 4. Both traces are run in code and checked against the GEMM.

14.1 The example

Weights (half are 0)

W	k0	k1	k2	k3
A	2	0	0	1
B	0	3	1	0

Spike train of token p on input k

train t0..t3	p0	p1	p2	p3
k0	1101	0100	0010	0000
k1	1101	0100	0010	0000
k2	0100	0000	1010	0001
k3	0000	0010	0000	0101

Output both designs must produce

out t0..t3	p0	p1	p2	p3
A	[2,2,0,2]	[0,2,1,0]	[0,0,2,0]	[0,1,0,1]
B	[3,4,0,3]	[0,3,0,0]	[1,0,4,0]	[0,0,0,1]

There are **17 spikes** in total. Every input row k has exactly one non-zero weight (A or B), so every spike needs exactly one useful addition.

14.2 The same spikes, stored two ways

Prosperity a **bit matrix of (tokens × T) rows × K columns**. One row = one token at one timestep, one bit per input. The row number is the label: row = T · p + t (the official simulator reshapes T × tokens × K to tokens × T × K). Nothing else is stored per row.

row	(token, t)	k0 k1 k2 k3
0	p0 t0	1100
1	p0 t1	1110
2	p0 t2	0000
3	p0 t3	1100
4	p1 t0	0000
5	p1 t1	1100
6	p1 t2	0001
7	p1 t3	0000
8	p2 t0	0010
9	p2 t1	0000
10	p2 t2	1110
11	p2 t3	0000
12	p3 t0	0000
13	p3 t1	0001
14	p3 t2	0000
15	p3 t3	0011

Real size: a tile is 256 rows (32 tokens × T = 8) × 16 inputs.

TIDES a **K × tokens matrix of T-bit trains**. One entry = all T timesteps of one token on one input; the whole train is used in one cycle by the T adders of a PE.

	p0	p1	p2	p3
k0	1101	0100	0010	0000
k1	1101	0100	0010	0000
k2	0100	0000	1010	0001
k3	0000	0010	0000	0101

Real size: an output tile is 64 filters × 128 tokens; a K-tile is 4 inputs (the 4 PE rows).

> **First difference.** Prosperity treats each timestep as a separate row and looks for *rows that repeat*. TIDES keeps the T timesteps of a token together and looks for *tokens whose spikes do not collide*, so several tokens can share one trip through the array.

14.3 Prosperity: reuse the result of a subset row

Prosperity's ProSparsity Processing Unit has four stages. All four run per 256 × 16 tile; here the tile is the whole 16 × 4 matrix.

- **Detector** (1 KB TCAM + popcounts): loads the tile into a TCAM, then for each row searches, in one cycle, for every row whose spikes are a subset of it (the row's 1s are masked to "don't care"). It also counts the 1s.
- **Pruner**: keeps one **prefix** per row, the subset with the most 1s (an identical row only if it comes earlier), and computes the **pattern = row XOR prefix**: the spikes still to be added. Rows with one spike are not searched; reusing them would save nothing.
- **Dispatcher** (1.5 KB product-sparsity table: prefix + pattern per row): stable-sorts the rows by number of 1s, so every prefix is issued before the rows that reuse it.
- **Processor** (128 adders, one per output channel): for each issued row, loads the prefix's result from the output buffer, then adds one whole **weight row W[k] (all n output channels)** per remaining spike, one spike per cycle. An exact match costs one cycle (copy). All-zero rows cost nothing.

issue	row	(token, t)	spikes	ones	prefix	pattern = row XOR prefix	what the Pruner found	Processor	[A,B]	steps
1	6	p1 t2	0001	1	-	0001	1 spike: not searched	0 + W[k3]	[1,0]	1
2	8	p2 t0	0010	1	-	0010	1 spike: not searched	0 + W[k2]	[0,1]	1
3	13	p3 t1	0001	1	-	0001	1 spike: not searched	0 + W[k3]	[1,0]	1
4	0	p0 t0	1100	2	-	1100	no subset row: computed fully	0 + W[k0] + W[k1]	[2,3]	2
5	3	p0 t3	1100	2	0	0000	**exact match** of row 0: copy	out[row 0]	[2,3]	1
6	5	p1 t1	1100	2	0	0000	**exact match** of row 0: copy	out[row 0]	[2,3]	1
7	15	p3 t3	0011	2	6	0010	**partial match**: row 6 + rest	out[row 6] + W[k2]	[1,1]	1
8	1	p0 t1	1110	3	0	0010	**partial match**: row 0 + rest	out[row 0] + W[k2]	[2,4]	1
9	10	p2 t2	1110	3	1	0000	**exact match** of row 1: copy	out[row 1]	[2,4]	1
								total (7 all-zero rows are free)		**10**

W[k0] = [**A** 2, **B** 0], W[k1] = [0, 3], W[k2] = [0, 1], W[k3] = [1, 0]. Every result was checked against the GEMM output in 14.1.

> **What Prosperity saves:** 17 spikes need only 10 steps, because 10 spikes are covered by prefix rows (rows 3, 5 and 10 repeat earlier rows; rows 1 and 15 extend one).
> **What it cannot save:** each step adds a whole weight row. Here half of every row is 0, so 7 of its 14 additions add 0. Pruning the weights does not remove a single step.

14.4 TIDES: pack non-zero weights offline, pack non-colliding tokens at run time

Offline (filter packing). A and B never have a non-zero weight on the same input, so they share **one column**. PE k keeps only its one non-zero weight and that filter's label. The zero weights are never stored or added.

column 0	filter	weight
PE k0	**A** (label 0)	2
PE k1	**B** (label 1)	3
PE k2	**B** (label 1)	1
PE k3	**A** (label 0)	1

Run time (TIP, in the pre-processing unit). Tokens are first-fit into groups of up to 4 such that, on every input, **at most one member spikes per timestep**. Token p1 cannot join p0: both spike on k0 at t1 (and on k1 at t1). p2 and p3 fit beside p0:

$$g1 = \{p0, p2, p3\}, \quad g2 = \{p1\}$$

Each lane t of a PE carries one bit plus the member label; the PE adds its weight to **slot = 4 · filter label + member label** (Section 13). The sums leave the column with no labels.

Group g1 = {m0 = p0, m1 = p2, m2 = p3}: one cycle through the column, all four lanes (timesteps) of every PE at once

PE	lane t0	lane t1	lane t2	lane t3
PE k0 (A, 2)	p0 → slot 0 **+2**	p0 → slot 0 **+2**	p2 → slot 1 **+2**	p0 → slot 0 **+2**
PE k1 (B, 3)	p0 → slot 4 **+3**	p0 → slot 4 **+3**	p2 → slot 5 **+3**	p0 → slot 4 **+3**
PE k2 (B, 1)	p2 → slot 5 **+1**	p0 → slot 4 **+1**	p2 → slot 5 **+1**	p3 → slot 6 **+1**
PE k3 (A, 1)	-	p3 → slot 2 **+1**	-	p3 → slot 2 **+1**

Group g2 = {m0 = p1}: one cycle through the column, all four lanes (timesteps) of every PE at once

PE	lane t0	lane t1	lane t2	lane t3
PE k0 (A, 2)	-	p1 → slot 0 **+2**	-	-
PE k1 (B, 3)	-	p1 → slot 4 **+3**	-	-
PE k2 (B, 1)	-	-	-	-
PE k3 (A, 1)	-	-	p1 → slot 0 **+1**	-

What TIDES saves: the whole GEMM is **2 steps** (2 groups × 1 pass) instead of 10. It never adds a zero weight, never spends a cycle on a silent token, and fills g1's lanes on k0, k1 and k2 completely (12 of 12 lanes).
What it cannot save: it has no reuse between rows, so it performs all 17 additions. Rows 3, 5 and 10, which Prosperity copied, are added again.

14.5 The example side by side

	Prosperity	TIDES
one step does	1 spike bit × a whole weight row (all n output channels)	1 packed group (≤ 4 tokens × all T timesteps) × the 4 inputs of the column
adders used per step here	n = 2 (one per output channel)	4 PEs × T = 4 lanes = 16
steps	**10** (7 remaining spike bits + 3 exact-match copies)	**2** (2 groups × 1 pass)
spike bits (of 17) not added again	10 (reused from prefix rows)	0
weight additions issued	14 (7 bits × 2 channels)	17
of which with a non-zero weight	7 (the other 7 add 0)	17 (zero weights are not stored)

14.6 The main difference

	Prosperity	TIDES
sparsity it exploits	spikes (bit sparsity) + repeated or nested rows (product sparsity)	spikes + **weights** (both packed)
weights	dense: every step adds a whole 128-wide weight row, zeros included	only non-zeros are stored, loaded and added
timesteps	T separate rows per token, matched against each other	all T in one cycle (T adders per PE)
cycles ≈	(remaining spike bits + exact matches) × ⌈N / 128⌉	Σ over (filter block, K-tile) of groups × ⌈packed columns / 4⌉
gets faster when	activations get sparser or more repetitive	activations get sparser **or weights get sparser**
wastes cycles on	zero weights (pruning does not help)	groups with few spikes: a pass costs one cycle per group, even if it carries one spike
per-row labels	none (row number = T·p + t); prefix + pattern per row in a 1.5 KB table	none on sums; filter label per weight, member label per spike bit
DRAM	dense 8-bit weights, re-read for every 256-row tile	packed non-zero weights only

14.7 The same counts on a real layer

BERT, real sentence, spikes chained from layer to layer (lm_chain.py, BN re-estimated). Both designs have 128 adders at 500 MHz.

BERT block 0, FC1 (768 → 3072), 25 tokens, T = 8		**90 % pruned**	**99 % pruned**
Prosperity 256 × 16 tiles, n = 128	spike bits (200 rows × 768)	7,065	3,822
	left after ProSparsity (+ exact-match rows)	4,274 (+115)	2,614 (+198)
	× 3072 / 128 = 24 channel tiles → steps	105,336	67,488
	cycles (official simulator)	**107,403**	**69,555**
	additions with a non-zero weight	1,311,765	80,859
	useful additions per cycle (of 128 adders)	12.2	1.2
TIDES 4 × 4 PEs, T adders, tiles 64 × 128	non-zero weights → packed columns	235,931 → 81,980	23,594 → 13,595
	(64-filter block, 4-input K-tile) pairs run → passes	9,216 → 23,896	8,417 → 8,436
	group steps (groups × passes)	183,839	38,396
	cycles (cycle model)	**189,268**	**41,608**
	additions with a non-zero weight (no reuse)	2,168,601	118,226
	useful additions per cycle (of 128 adders)	11.5	2.8
TIDES speed vs Prosperity		**0.57×**	**1.67×**

Reading the table. At 90 % pruning both designs keep about 12 of their 128 adders busy with useful work, but Prosperity needs 1.65× fewer additions because of reuse, so it is faster. Going to 99 % pruning, Prosperity's steps fall only because the spikes fall (7,065 → 3,822 bits); its weight rows are now 99 % zeros, so only 1.2 useful additions happen per cycle. TIDES's packed columns fall 6.0× (81,980 → 13,595) and it overtakes. In the whole-model BERT run the crossover is at about 97 % pruning (64 GB/s DRAM), and lower when DRAM is slower, because Prosperity re-reads dense weights.